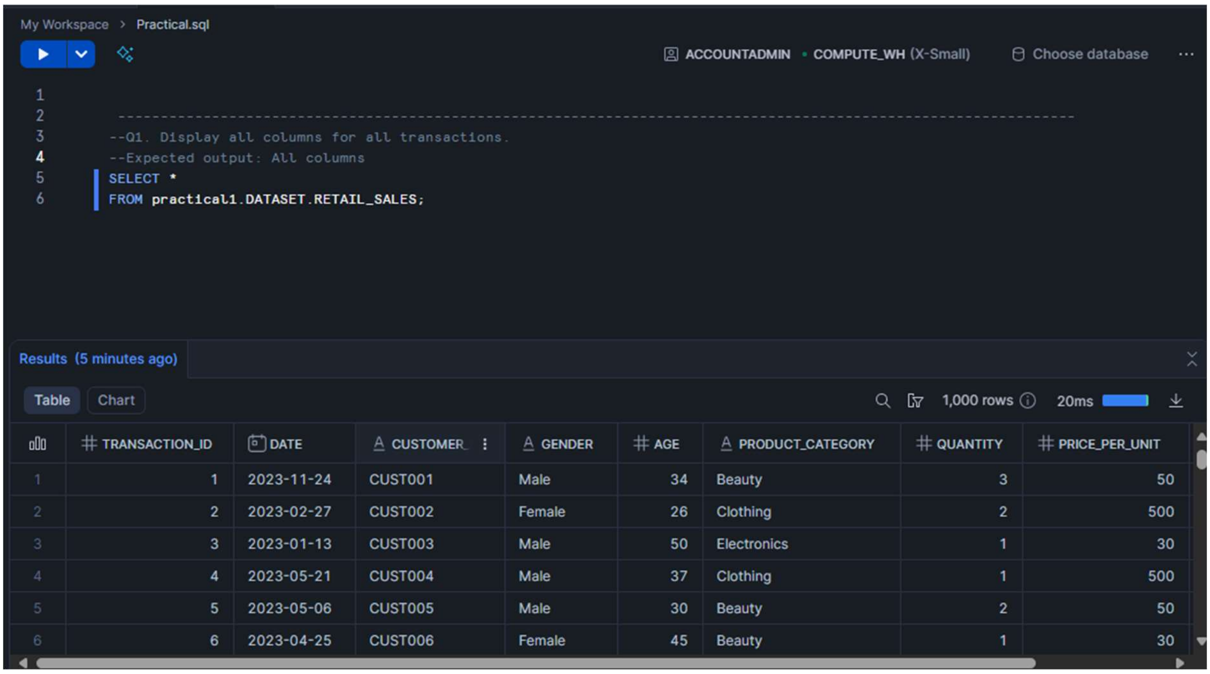
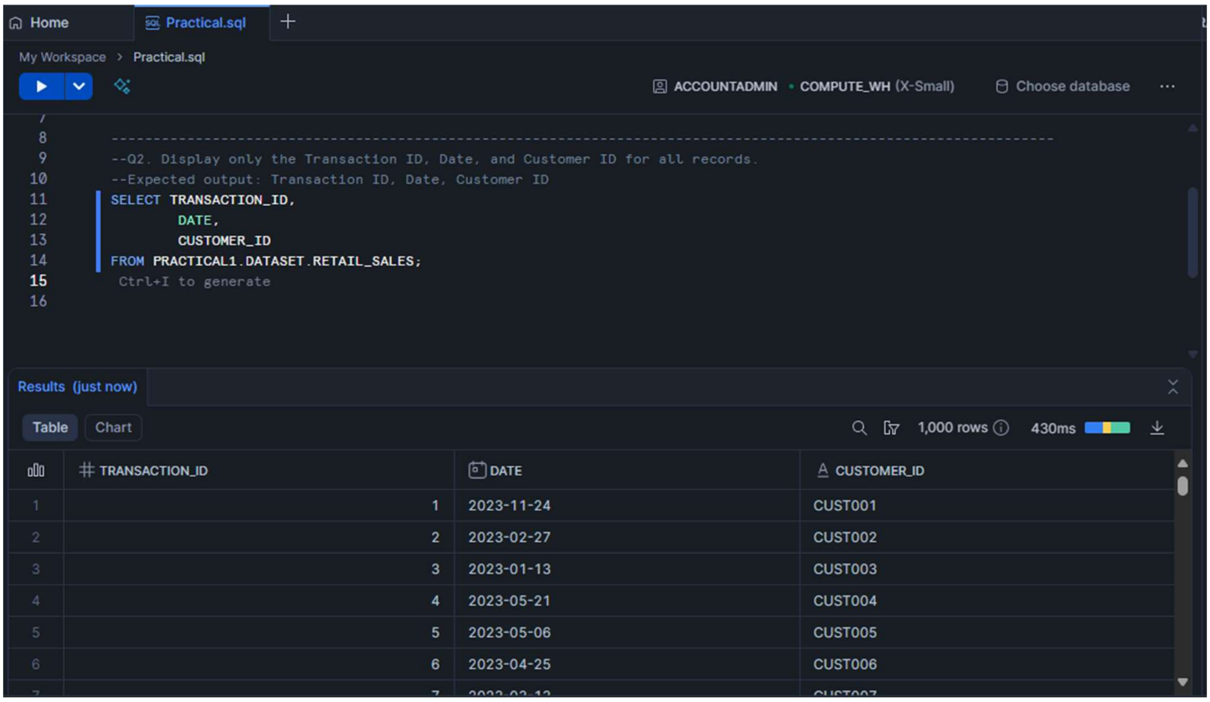
Tebogo Masilo

Practical 1 : Basic SQL Syntax

Question 1



Question 2



Question 3

My Workspace > Practical.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small)   Choose database

```sql
------------------------------------------------------------------------------------------------
---Q3. Display all the distinct product categories in the dataset.
--Expected output: Product Category
SELECT DISTINCT PRODUCT_CATEGORY
FROM PRACTICAL1.DATASET.RETAIL_SALES;
 Ctrl+I to generate
```

Results (just now)

Table   Chart   3 rows   86ms

	PRODUCT_CATEGORY
1	Clothing
2	Beauty
3	Electronics

Question 4

My Workspace > Practical.sql

ACCOUNTADMIN • COMPUTE_WH (X-Small)   Choose database

```sql
------------------------------------------------------------------------------------------------
---Q4. Display all the distinct gender values in the dataset.
--Expected output: Gender

SELECT DISTINCT GENDER
FROM PRACTICAL1.DATASET.RETAIL_SALES;
```

Results (just now)

Table   Chart   2 rows   74ms

	GENDER
1	Male
2	Female

Question 5

```sql
--Q5. Display all transactions where the Age is greater than 40.
--Expected output: All columns
SELECT*
FROM PRACTICAL1.DATASET.RETAIL_SALES
WHERE AGE>40;
```

lts (10 hours ago)

ble | Chart

534 rows 69ms

TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY
3	2023-01-13	CUST003	Male	50	Electronics	1
6	2023-04-25	CUST006	Female	45	Beauty	1
7	2023-03-13	CUST007	Male	46	Clothing	2
9	2023-12-13	CUST009	Male	63	Electronics	2
10	2023-10-07	CUST010	Female	52	Clothing	4
14	2023-01-17	CUST014	Male	64	Clothing	4

Question 6

```sql
---------------------------------------------------------------------------------------------------
--Q6. Display all transactions where the Price per Unit is between 100 and 500.
---Expected output: All columns
SELECT*
FROM PRACTICAL1.DATASET.RETAIL_SALES
WHERE price_per_unit BETWEEN 100 AND 500;
```

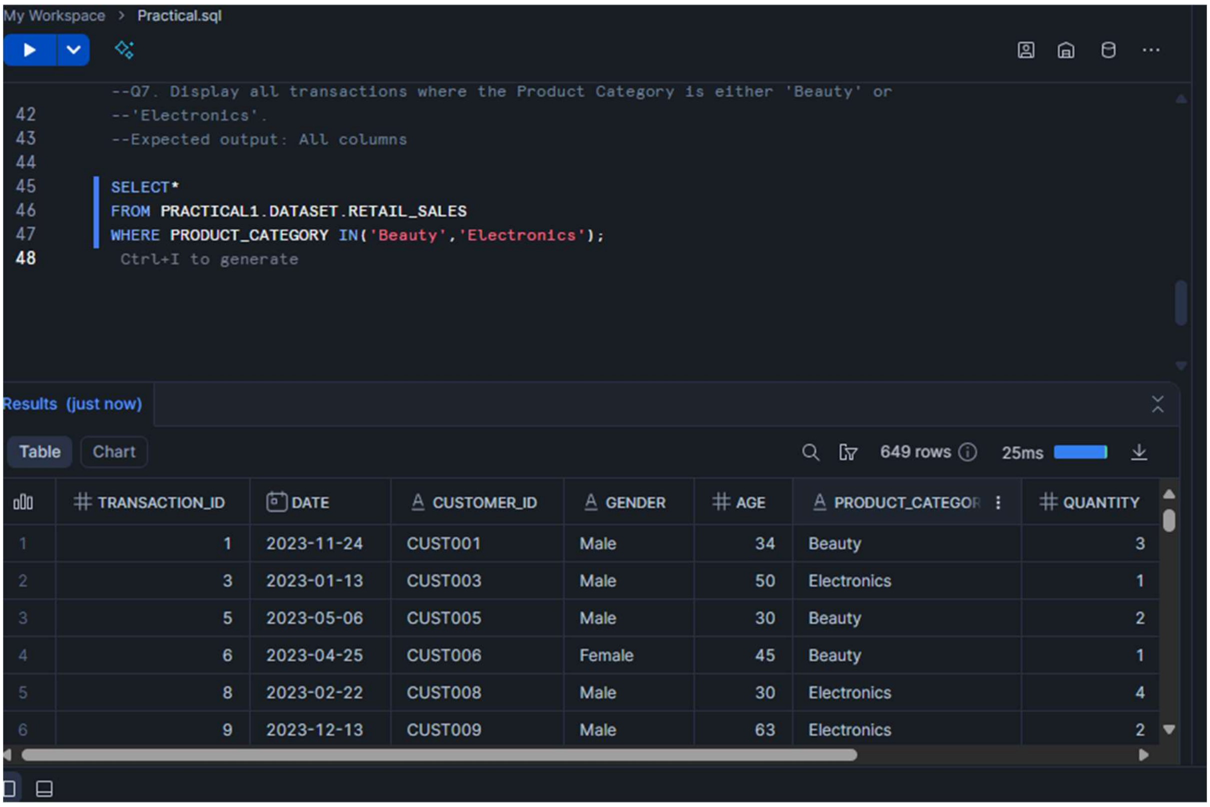
Results (1 minute ago)

Table | Chart

396 rows 1.3s

	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY	PRICE_PER_UNIT	TOTAL_AI
1	CUST002	Female	26	Clothing	2	500	
2	CUST004	Male	37	Clothing	1	500	
3	CUST009	Male	63	Electronics	2	300	
4	CUST013	Male	22	Electronics	3	500	
5	CUST015	Female	42	Electronics	4	500	
6	CUST016	Male	19	Clothing	3	500	

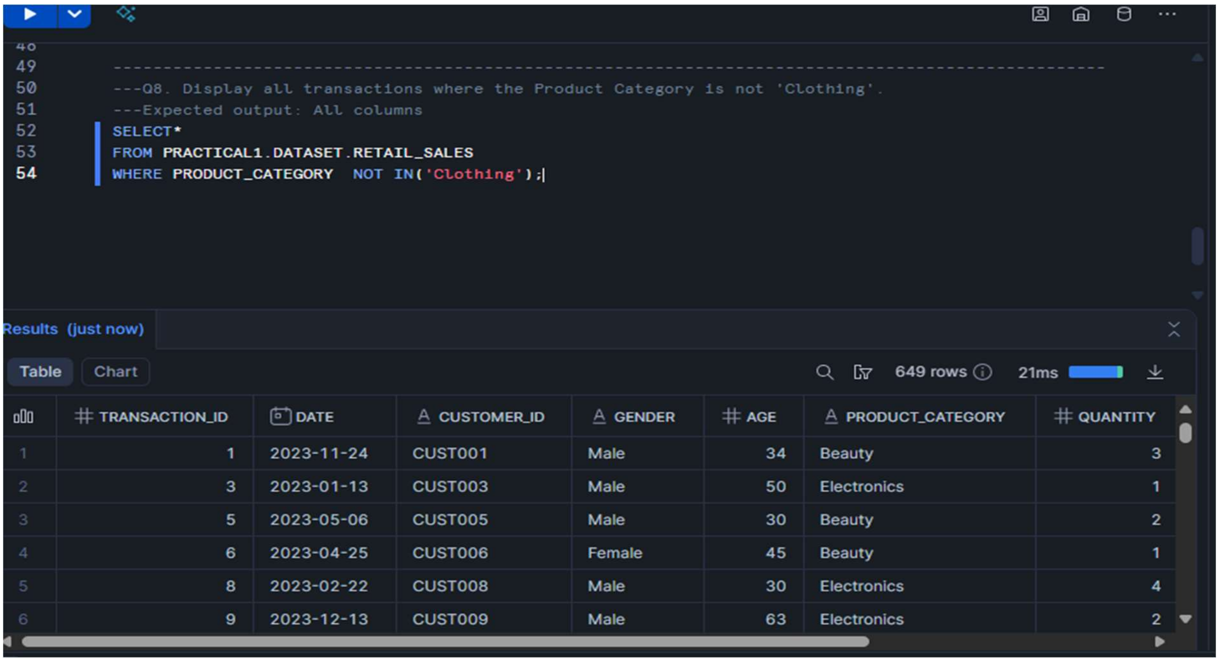
Question 7




```sql
--Q7. Display all transactions where the Product Category is either 'Beauty' or
--'Electronics'.
--Expected output: All columns

SELECT*
FROM PRACTICAL1.DATASET.RETAIL_SALES
WHERE PRODUCT_CATEGORY IN('Beauty','Electronics');
 Ctrl+I to generate
```

	TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGOR	QUANTITY
1	1	2023-11-24	CUST001	Male	34	Beauty	3
2	3	2023-01-13	CUST003	Male	50	Electronics	1
3	5	2023-05-06	CUST005	Male	30	Beauty	2
4	6	2023-04-25	CUST006	Female	45	Beauty	1
5	8	2023-02-22	CUST008	Male	30	Electronics	4
6	9	2023-12-13	CUST009	Male	63	Electronics	2

Question 8




```sql
------------------------------------------------------------------------------------------------
---Q8. Display all transactions where the Product Category is not 'Clothing'.
---Expected output: All columns
SELECT*
FROM PRACTICAL1.DATASET.RETAIL_SALES
WHERE PRODUCT_CATEGORY  NOT IN('Clothing');
```

	TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY
1	1	2023-11-24	CUST001	Male	34	Beauty	3
2	3	2023-01-13	CUST003	Male	50	Electronics	1
3	5	2023-05-06	CUST005	Male	30	Beauty	2
4	6	2023-04-25	CUST006	Female	45	Beauty	1
5	8	2023-02-22	CUST008	Male	30	Electronics	4
6	9	2023-12-13	CUST009	Male	63	Electronics	2

Question 9

```sql
---------------------------------------------------------------------------------------------------
--Q9. Display all transactions where the Quantity is greater than or equal to 3.
---Expected output: All columns
SELECT*
FROM PRACTICAL1.DATASET.RETAIL_SALES
WHERE QUANTITY >=3;
| Ctrl+I to generate
```

Results (just now)

Table Chart 504 rows 24ms

	TRANSACTION_ID	DATE	CUSTOMER_ID	GENDER	AGE	PRODUCT_CATEGORY	QUANTITY
1	1	2023-11-24	CUST001	Male	34	Beauty	3
2	8	2023-02-22	CUST008	Male	30	Electronics	4
3	10	2023-10-07	CUST010	Female	52	Clothing	4
4	12	2023-10-30	CUST012	Male	35	Beauty	3
5	13	2023-08-05	CUST013	Male	22	Electronics	3
6	14	2023-01-17	CUST014	Male	64	Clothing	4

Question 10

```sql
---------------------------------------------------------------------------------------------------
---Q10. Count the total number of transactions.
---Expected output: Total_Transactions
SELECT COUNT(TRANSACTION_ID) AS Total_Transcations
FROM PRACTICAL1.DATASET.RETAIL_SALES;
```

Results (just now)

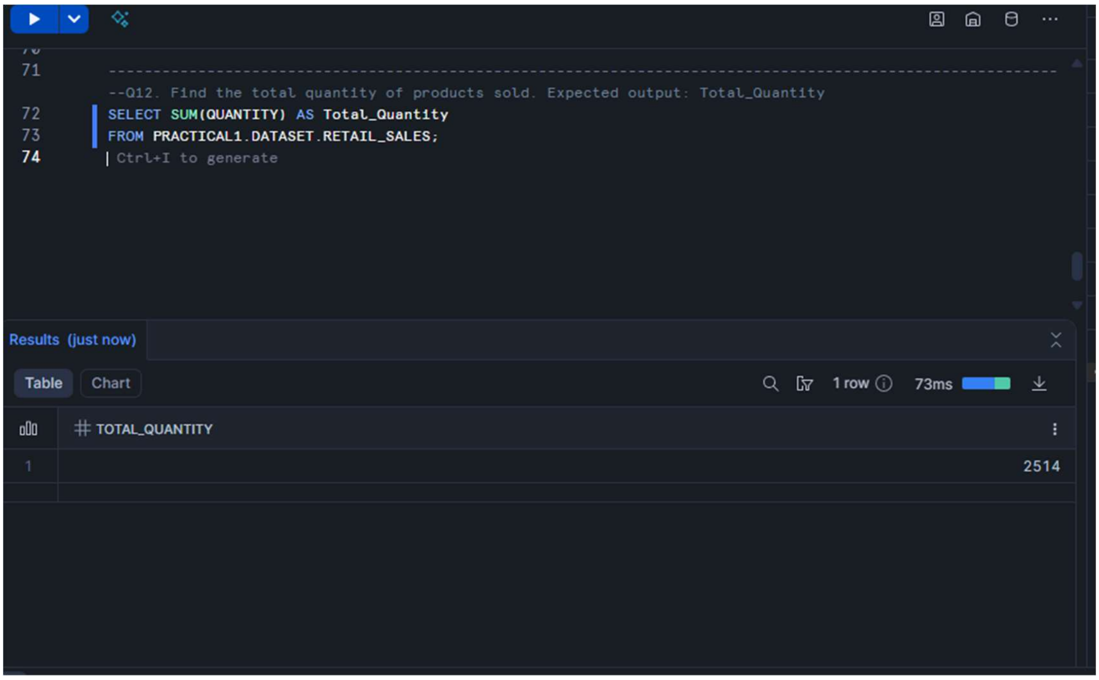
Table Chart 1 row 35ms

	TOTAL_TRANSCATIONS
1	1000

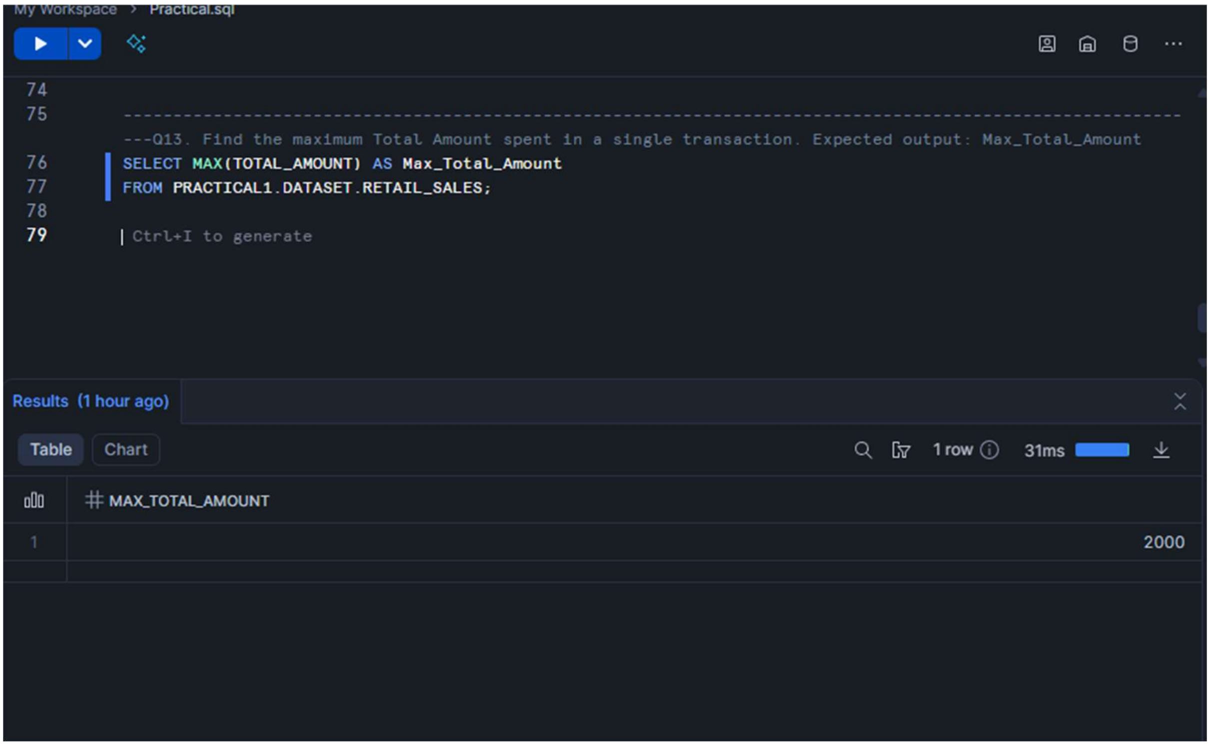
\

Question 11

Question 12

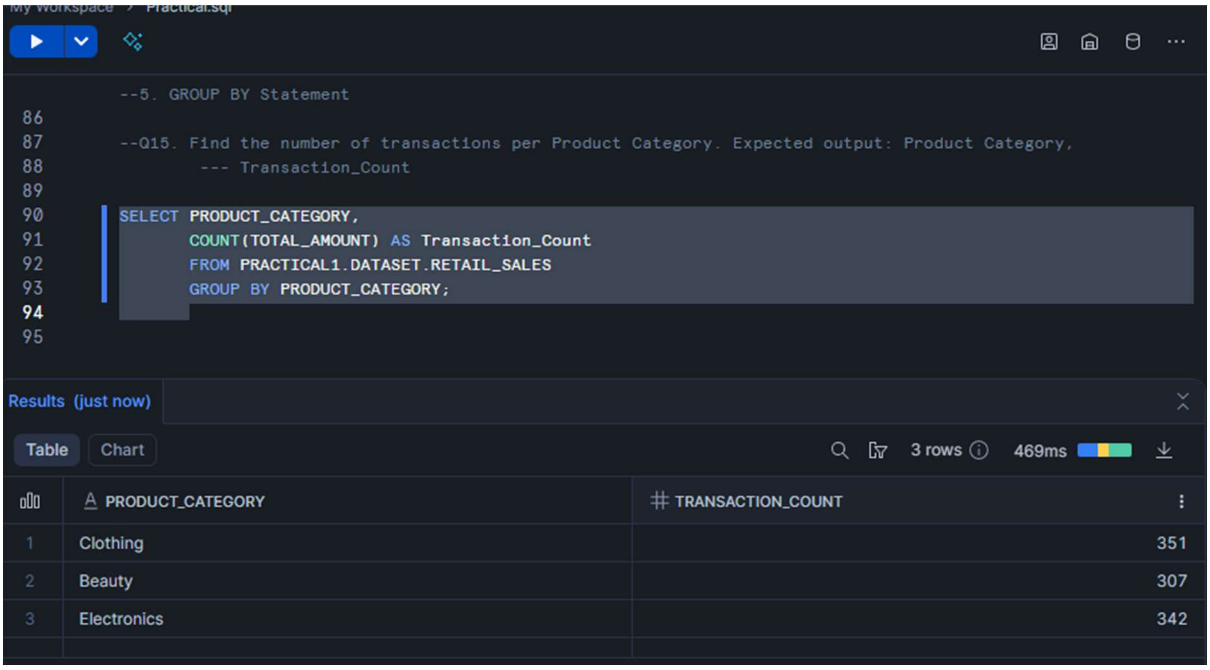


Question 13



Question 14

Question 15



My Workspace > Practical.sql

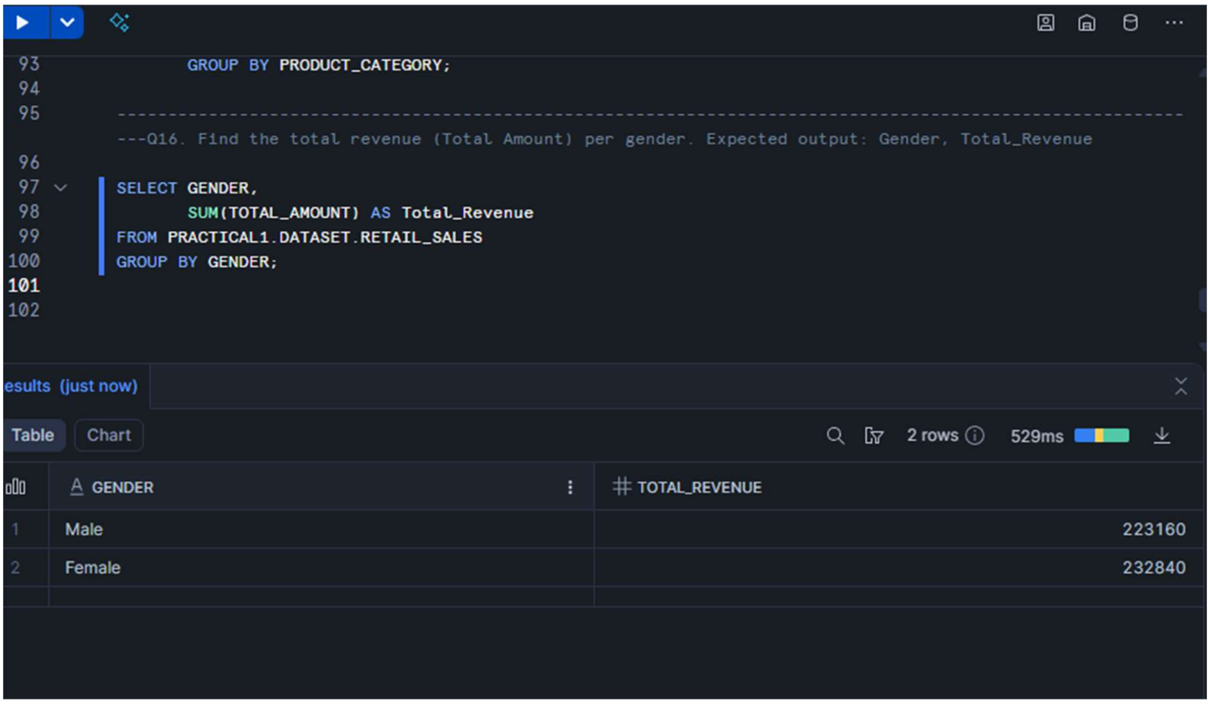
```sql
--5. GROUP BY Statement

--Q15. Find the number of transactions per Product Category. Expected output: Product Category,
 --- Transaction_Count

SELECT PRODUCT_CATEGORY,
       COUNT(TOTAL_AMOUNT) AS Transaction_Count
       FROM PRACTICAL1.DATASET.RETAIL_SALES
       GROUP BY PRODUCT_CATEGORY;
```

Results (just now)

Table Chart 3 rows 469ms

	PRODUCT_CATEGORY	TRANSACTION_COUNT
1	Clothing	351
2	Beauty	307
3	Electronics	342

Question 16



```sql
       GROUP BY PRODUCT_CATEGORY;

-----------------------------------------------------------------------------------------------------------
---Q16. Find the total revenue (Total Amount) per gender. Expected output: Gender, Total_Revenue

SELECT GENDER,
       SUM(TOTAL_AMOUNT) AS Total_Revenue
FROM PRACTICAL1.DATASET.RETAIL_SALES
GROUP BY GENDER;
```

esults (just now)

Table Chart 2 rows 529ms

	GENDER	TOTAL_REVENUE
1	Male	223160
2	Female	232840

QUESTION 17

My Workspace > Practical.sql

```sql
----------------------------------------------------------------------------------------
--Q17. Find the average Price per Unit per product category. Expected output: Product Category, Average_Price

SELECT PRODUCT_CATEGORY,
       AVG(PRICE_PER_UNIT) AS Average_Price
FROM PRACTICAL1.DATASET.RETAIL_SALES
GROUP BY PRODUCT_CATEGORY;
```

Results (just now)

Table | Chart — 3 rows — 87ms

	PRODUCT_CATEGORY	AVERAGE_PRICE
1	Beauty	184.055375
2	Clothing	174.287749
3	Electronics	181.900585

QUESTION 18

My Workspace > Practical.sql

```sql
----------------------------------------------------------------------------------------
--6. HAVING Clause
---Q18. Find the total revenue per product category where total revenue is greater than 10,000. Expected output: Product Category, Total_Revenue
SELECT PRODUCT_CATEGORY,
       SUM(TOTAL_AMOUNT) AS Total_Revenue
    FROM PRACTICAL1.DATASET.RETAIL_SALES
    GROUP BY PRODUCT_CATEGORY
    HAVING Total_Revenue > 10000;

----------------------------------------------------------------------------------------
```

Results (just now)

Table | Chart — 3 rows — 88ms

	PRODUCT_CATEGORY	TOTAL_REVENUE
1	Beauty	143515
2	Clothing	155580
3	Electronics	156905

QUESTION 19

My Workspace > Practical.sql

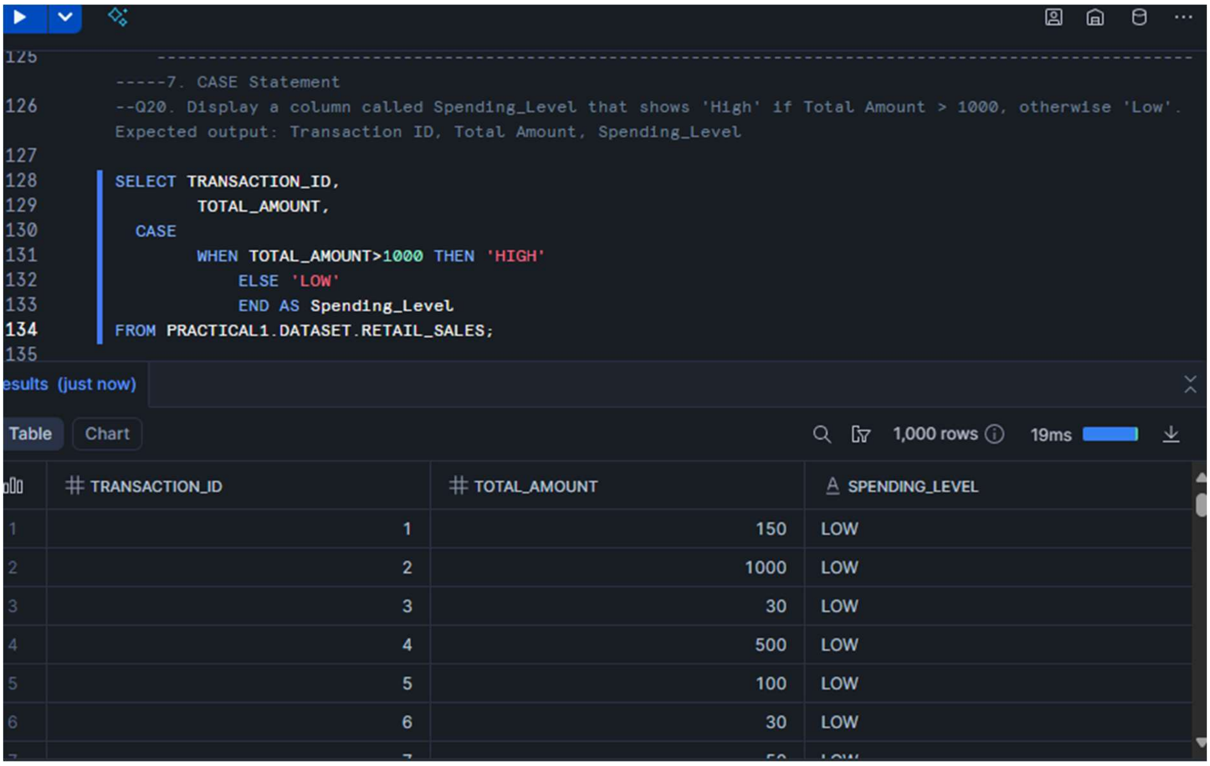
```sql
--------------------------------------------------------------------------------------------
-
--Q19. Find the average quantity per product category where the average is more than 2. Expected output:
  Product Category, Average_Quantity
   SELECT PRODUCT_CATEGORY,
          AVG(QUANTITY) AS Average_Quantity
  FROM PRACTICAL1.DATASET.RETAIL_SALES
  GROUP BY PRODUCT_CATEGORY
  HAVING Average_Quantity > 2;
```

Results (just now)

Table Chart 3 rows 765ms

	PRODUCT_CATEGORY	AVERAGE_QUANTITY
1	Beauty	2.511401
2	Clothing	2.547009
3	Electronics	2.482456

QUESTION 20



```sql
--------------------------------------------------------------------------------------------
-----7. CASE Statement
--Q20. Display a column called Spending_Level that shows 'High' if Total Amount > 1000, otherwise 'Low'.
Expected output: Transaction ID, Total Amount, Spending_Level

SELECT TRANSACTION_ID,
        TOTAL_AMOUNT,
   CASE
        WHEN TOTAL_AMOUNT>1000 THEN 'HIGH'
            ELSE 'LOW'
            END AS Spending_Level
FROM PRACTICAL1.DATASET.RETAIL_SALES;
```

esults (just now)

Table Chart 1,000 rows 19ms

	TRANSACTION_ID	TOTAL_AMOUNT	SPENDING_LEVEL
1	1	150	LOW
2	2	1000	LOW
3	3	30	LOW
4	4	500	LOW
5	5	100	LOW
6	6	30	LOW

QUESTION 21

```sql
--Q21. Display a new column called Age_Group that labels customers as:
--·
--'Youth' if Age < 30·
--'Adult' if Age is between 30 and 59·
--'Senior' if Age >= 60
--Expected output: Customer ID, Age, Age_Group
SELECT CUSTOMER_ID,
       AGE,
       CASE
       WHEN AGE < 30 THEN 'YOUTH'
       WHEN AGE BETWEEN 30 AND 59 THEN 'Adult'
       ELSE 'SENIOR'
       END AS AGE_GROUP
FROM PRACTICAL1.DATASET.RETAIL_SALES;
```

Results (1 minute ago)

Download results

Table Chart

1,000 rows 67ms

	CUSTOMER_ID	AGE	AGE_GROUP
18	CUST018	47	Adult
19	CUST019	62	SENIOR
20	CUST020	22	YOUTH
21	CUST021	50	Adult